KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

(AUTONOMOUS)

NAMAKKAL- TRICHY MAIN ROAD, THOTTIAM, TRICHY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

20IT502 – WEB TECHNOLOGY

HANDOUTS

VI SEMESTER

UNIT I WEB SITE BASICS AND HTML

Web Essentials.

Client:

- Applications that run on computers
- Rely on servers for
    - Files
- Devices
- Processing power

Computers or processes that manage network resources

- Disk drives (file servers)
- Printers (print servers)
- Network traffic (network servers)

Process takes place

- on the server
- on the client

Servers

- Store and protect data
- Process requests from clients

Clients

- Make requests
- Format data on the desktop

Client-side Environment

- The client-side environment used to run scripts is usually a browser. The processing takes place on the end users computer. The source code is transferred from the web server to the user's computer over the internet and run directly in the browser.

- The scripting language needs to be enabled on the client computer. Sometimes if a user is conscious of security risks they may switch the scripting facility off. When this is the case a message usually pops up to alert the user when script is attempting to run.

Server-side Environment

- The server-side environment that runs a scripting language is a web server. A user's

request is fulfilled by running a script directly on the web server to generate dynamic HTML pages. This HTML is then sent to the client browser. It is usually used to provide interactive web sites that interface to databases or other data stores on the server.

- This is different from client-side scripting where scripts are run by the viewing web browser, usually in JavaScript. The primary advantage to server-side scripting is the ability to highly customize the response based on the user's requirements, access rights, or queries into data stores.

- Server side scripting is used to create dynamic pages based a number of conditions when the users browser makes a request to the server.

- Client side scripting is used when the users browser already has all the code and the page is altered on the basis of the users input.

-

- Server executes server-side scripts to send out a page but it does not execute client-side scripts.

- List of web servers,

- Apache

- IIS

Transmission Control Protocol (TCP)

A protocol is a set of rules and guidelines for communicating data. Rules are defined for each step and process during communication between two or more computers. Networks have to follow these rules to successfully transmit data.

TCP is a connection oriented protocol and offers end-to-end packet delivery. It acts as back bone for connection. It exhibits the following key features:

- Transmission Control Protocol (TCP) corresponds to the Transport Layer of OSI Model.
- TCP is a reliable
- TCP offers:
- Stream Data Transfer.
- Reliability.
- Efficient Flow Control
- Full-duplex operation.
- Multiplexing.

TCP ensures reliability by sequencing bytes with a forwarding acknowledgement number that indicates to the destination the next byte the source expect to receive. It retransmits the bytes, If it

not receive acknowledged with in specified time period.

Internet_Protocol (IP)

Internet_Protocol is connectionless and unreliable protocol. It ensures no guarantee of successfully transmission of data. Internet Protocol (IP) corresponds to the Network Layer of OSI Model. In order to make it reliable, it must be paired with reliable protocol such as TCP at the transport layer. Internet protocol transmits the data in form of a datagram.

User Datagram Protocol (UDP)

Like IP, UDP is connectionless and unreliable protocol. It doesn't require making a connection with the host to exchange data. Since UDP is unreliable protocol, there is no mechanism for ensuring that data sent is received. UDP transmits the data in form of a datagram. The UDP datagram consists of five parts as shown in the following diagram:

Source Port	Destination Port
Length	UDP checksum
Data	

File Transfer Protocol (FTP)

FTP is used to copy files from one host to another. FTP offers the mechanism for the same in following manner:

FTP creates two processes such as Control Process and Data Transfer Process at both ends i.e. at client as well as at server. FTP establishes two different connections: one is for data transfer and other is for control information. Control connection is made between control processes while Data Connection is made between data processes. FTP uses port 21 for the control connection and Port 20 for the data connection.

TELNET

Telnet is a protocol used to log in to remote computer on the internet. There are a number of Telnet clients having user friendly user interface. The following diagram shows a person is logged in to computer A, and from there, he remote logged into computer B.

HTTP

HTTP is a communication protocol. It defines mechanism for communication between browser and the web server. It is also called request and response protocol because the communication between browser and server takes place in request and response pairs.

HTTP Request

- Request line
- Header Fields

- Message body

HTTP Response

- Status line
- Headers
- Message body

HTTP Request Header:

EXAMPLE

HTTP Method:

• HTTP method is supplied in the request line and specifies the operation that the client has requested.

Some common methods:

• Options

• Get

• Head

• Post

• Put

• Move

• Delete

· **Markup Languages:**

· HTML is the standard markup language for creating Web pages.

· HTML stands for Hyper Text Markup Language

· HTML describes the structure of Web pages using markup

· HTML elements are the building blocks of HTML pages

· HTML elements are represented by tags

· HTML tags label pieces of content such as "heading", "paragraph", "table", and so on

· Browsers do not display the HTML tags, but use them to render the content of the page

· HTML tags are used to mark-up HTML elements

· HTML tags are surrounded by the two characters < and >

- The surrounding characters are called **angle brackets**.
- HTML tags normally come in pairs like **** and ****
- The first tag in a pair is the start tag, the second tag is the end tag
- The text between the start and end tags is the element content
- HTML tags are not case sensitive, **means <B> the same as **
- **<tagname>content goes here...</tagname>**
- **HTML Page Structure**

```
<html>
  <head>
    <title>Page title</title>
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>This is a paragraph.</p>
    <p>This is another paragraph.</p>
  </body>
</html>
```

-
- **Some Fundamental HTML Elements:**
-

Tag	Description
<!DOCTYPE>	**Defines the document type**
<html>	**Defines an HTML document**
<head>	**Defines information about the document**
<title>	**Defines a title for the document**
<body>	**Defines the document's body**

<h1> to <h6>	Defines HTML headings
<p>	Defines a paragraph
 	Inserts a single line break
<hr>	Defines a thematic change in the content
<!-- ... -->	Defines a comment

·

· **Tables Tags**

·

Tag	Description
<table>	Defines a table
<caption>	Defines a table caption
<th>	Defines a header cell in a table
<tr>	Defines a row in a table
<td>	Defines a cell in a table
<thead>	Groups the header content in a table
<tbody>	Groups the body content in a table

·

<tfoot>	Groups the footer content in a table
<col>	Specifies column properties for each column within a <colgroup> element
<colgroup>	Specifies a group of one or more columns in a table for formatting

· **Audio / Video Tags**

·

Tag	Description
<audio>	Defines sound content
<source>	Defines multiple media resources for media elements (<video>, <audio> and <picture>)
<track>	Defines text tracks for media elements (<video> and <audio>)
<video>	Defines a video or movie

Frames Tags

Tag	Description
<frame>	Not supported in HTML5. Defines a window (a frame) in a frameset
<frameset>	Not supported in HTML5. Defines a set of frames
<noframes>	Not supported in HTML5. Defines an alternate content for users that do not support frames
<iframe>	Defines an inline frame

Lists Tags:

Tag	Description
<ul>	Defines an unordered list
<ol>	Defines an ordered list

<li>	**Defines a list item**
<dir>	**Not supported in HTML5. Use <ul> instead.** **Defines a directory list**
<dl>	**Defines a description list**
<dt>	**Defines a term/name in a description list**
<dd>	**Defines a description of a term/name in a description list**

Examples:

```
<!DOCTYPE html>
<html>
<body>
<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
<h4>This is heading 4</h4>
<h5>This is heading 5</h5>
<h6>This is heading 6</h6>
</body>
</html>
```

This is heading 1

This is heading 2

This is heading 3

This is heading 4

This is heading 5

This is heading 6

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>Width and Height Attributes</h2>
<p>The width and height attributes of the img tag, defines the width and height of the image:</p>
<img src="img.jpg" width="500" height="600">
</body>
</html>
```

HTML Styles

```
<!DOCTYPE html>
<html>
<body>
<p>I am normal</p>
<p style="color:red;">I am red</p>
<p style="color:blue;">I am blue</p>
<p style="font-size:50px;">I am big</p>
</body>
</html>
```

I am normal

I am red

I am blue

I am big

```
<!DOCTYPE html>
<html>
<body>
<h2>The href Attribute</h2>
<p>HTML links are defined with the a tag. The link address is specified in the href attribute:</p>
<a href="https://www.w3schools.com">Visit W3Schools</a>
</body>
</html>
```

The href Attribute

HTML links are defined with the a tag. The link address is specified in the href attribute:

Visit W3Schools

Forms:

```
<!DOCTYPE html>
```

```
<html>
<body>
<h2>HTML Forms</h2>
<form action="/action_page.php">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>
<p>If you click the "Submit" button, the form-data will be sent to a page called "/action_page.php".</p>
</body>
</html>
```

The HTML <form> Elements

The HTML <form> element can contain one or more of the following form elements:

- <input>
- <label>
- <select>
- <textarea>
- <button>
- <fieldset>
- <legend>
- <datalist>
- <output>
- <option>
- <optgroup>

HTML contains several elements for defining text with a special meaning.

```
<!DOCTYPE html>
<html>
<body>
<p><b>This text is bold</b></p>
<p><i>This text is italic</i></p>
<p>This is<sub> subscript</sub> and <sup>superscript</sup></p>
</body>
</html>
```

Output:
This text is bold
This text is italic
This is subscript and superscript

```
<!DOCTYPE html>
<html>
<body>
<h1 style="background-color:Tomato;">Tomato</h1>
<h1 style="background-color:Orange;">Orange</h1>
<h1 style="background-color:DodgerBlue;">DodgerBlue</h1>
<h1 style="background-color:MediumSeaGreen;">MediumSeaGreen</h1>
<h1 style="background-color:Gray;">Gray</h1>
<h1 style="background-color:SlateBlue;">SlateBlue</h1>
<h1 style="background-color:Violet;">Violet</h1>
<h1 style="background-color:LightGray;">LightGray</h1>
</body>
</html>
```

```
<!DOCTYPE html>
<html>
<style>
table, th, td {
  border:1px solid black;
}
</style>
<body>
<h2>A basic HTML table</h2>
<table style="width:100%">
  <tr>
    <th>Company</th>
    <th>Contact</th>
    <th>Country</th>
  </tr>
  <tr>
    <td>Alfreds Futterkiste</td>
    <td>Maria Anders</td>
    <td>Germany</td>
```

```
  </tr>
tr>
    <td>Centro comercial Moctezuma</td>
    <td>Francisco Chang</td>
    <td>Mexico</td>
  </tr>
</table>
<p>To understand the example better, we have added borders to the table.</p>
</body>
</html>

<!DOCTYPE html>
<html>
<head>
<style>
table, th, td {
  border: 1px solid white;
  border-collapse: collapse;
}
th, td {
  background-color: #96D4D4;
}
</style>
</head>
<body>
<h2>Table With Invisible Borders</h2>
<p>Style the table with white borders and a background color of the cells to
make the impression of invisible borders.</p>
table style="width:100%">
  <tr>
    <th>Firstname</th>
    <th>Lastname</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Jill</td>
    <td>Smith</td>
    <td>50</td>
```

```
  </tr>
  <tr>
    <td>Eve</td>
    <td>Jackson</td>
    <td>94</td>
  </tr>
  <tr>
    <td>John</td>
    <td>Doe</td>
    <td>80</td>
  </tr>
</table>
</body>
</html>
```

HTML – Frames:

HTML frames are used to divide your browser window into multiple sections where each section can load a separate HTML document. A collection of frames in the browser window is known as a frameset. The window is divided into frames in a similar way the tables are organized: into rows and columns.

Creating Frames

To use frames on a page we use <frameset> tag instead of <body> tag. The <frameset> tag defines, how to divide the window into frames. The **rows** attribute of <frameset> tag defines horizontal frames and **cols** attribute defines vertical frames. Each frame is indicated by <frame> tag and it defines which HTML document shall open into the frame.

```
<!DOCTYPE html>
<html>
   <head>
      <title>HTML Frames</title>
   </head>

   <frameset rows = "10%,80%,10%">
      <frame name = "top" src = "/html/top_frame.htm" />
      <frame name = "main" src = "/html/main_frame.htm" />
      <frame name = "bottom" src = "/html/bottom_frame.htm" />

<body>Your browser does not support frames.</body>

   </frameset>
```

```
</html>

<!DOCTYPE html>
<html>

  <head>
    <title>HTML Frames</title>
  </head>

  <frameset cols = "25%,50%,25%">
    <frame name = "left" src = "/html/top_frame.htm" />
    <frame name = "center" src = "/html/main_frame.htm" />
    <frame name = "right" src = "/html/bottom_frame.htm" />

    <noframes>
      <body>Your browser does not support frames.</body>
    </noframes>
  </frameset>

</html>
```

What is XHTML?

XHTML stands for E**X**tensible **H**yper**T**ext **M**arkup **L**anguage

XHTML is a stricter, more XML-based version of HTML

XHTML is HTML defined as an XML application

XHTML is supported by all major browsers

Why XHTML?

XML is a markup language where all documents must be marked up correctly (be "well-formed").

XHTML was developed to make HTML more extensible and flexible to work with other data formats (such as XML). In addition, browsers ignore errors in HTML pages, and try to display the website even if it has some errors in the markup. So XHTML comes with a much stricter error handling.

HTML5:

HTML stands for Hyper Text Markup Language. It is used to design web pages using a markup language. HTML is an abbreviation of Hypertext and Markup language. Hypertext defines the link between the web pages.

The markup language is used to define the text document within the tag which defines the structure of web pages.

HTML 5 is the fifth and current version of HTML. It has improved the markup available for documents and has introduced application programming interfaces (API) and Document Object Model (DOM).

Features:

It has introduced new multimedia features which supports both audio and video controls by using <audio> and <video> tags.

There are new graphics elements including vector graphics and tags.

Enrich semantic content by including <header> <footer>, <article>, <section> and <figure> are added.

Drag and Drop- The user can grab an object and drag it further dropping it to a new location.

Geo-location services- It helps to locate the geographical location of a client.

Web storage facility which provides web application methods to store data on the web browser.

Uses SQL database to store data offline.

<article>

<aside>

<figcaption>

<figure>

<header>

<footer>

<main>

<mark>

<summary>

<section>

<nav>

<time>

<wbr>

<output>

<svg>: It is the Scalable Vector Graphics.

<canvas>:

<audio>: It defines the music or audio content.

<embed>: Defines containers for external applications (usually a video player).

<source>: It defines the sources for <video> and <audio>.

<track>: It defines the tracks for <video> and <audio>.

<video>: It defines the video content.

UNIT II CSS AND CLIENT SIDE SCRIPTING

Style Sheets: CSS-Introduction to Cascading Style Sheets:

CSS Syntax and Selectors

CSS Syntax: A CSS rule-set consists of a selector and a declaration block:

The selector points to the HTML element you want to style. The declaration block contains one or more declarations separated by semicolons. Each declaration includes a CSS property name and a value, separated by a colon.

A CSS declaration always ends with a semicolon, and declaration blocks are surrounded by curly braces.In the following example all <p> elements will be center-aligned, with a red text color:

Example:

```
p {
   color: red;
   text-align: center;
}
```

Types to attaching CSS with HTML:

a) External CSS
b) Internal CSS
c) Inline CSS

ii. Types of CSS selectors with example.

CSS Selectors: CSS selectors are used to "find" (or select) HTML elements based on their element name, id, class, attribute, and more.

The element Selector

The element selector selects elements based on the element name.You can select all <p> elements on a page like this (in this case, all <p>elements will be center-aligned, with a red text color):

Example:

```
p {
   text-align: center;
   color: red;
}
```

The id selector: uses the id attribute of an HTML element to select a specific element. The id of an element should be unique within a page, so the id selector is used to select one unique element!

To select an element with a specific id, write a hash (#) character, followed by the id of the element. The style rule below will be applied to the HTML element with id="para1":
Example:

```
#para1 {
   text-align: center;
   color: red;
}
```

Note: An id name cannot start with a number!

The class Selector: The class selector selects elements with a specific class attribute. To select elements with a specific class, write a period (.) character, followed by the name of the class. In the example below, all HTML elements with class="center" will be red and center-aligned:
Example

```
.center {
   text-align: center;
   color: red;
}
```

You can also specify that only specific HTML elements should be affected by a class.In the example below, only <p> elements with class="center" will be center-aligned:
Example

```
p.center {
   text-align: center;
   color: red;
}
```

HTML elements can also refer to more than one class.In the example below, the <p> element will be styled according to class="center" and to class="large":

```
<p class="center large">This paragraph refers to two classes.</p>
```

Note: A class name cannot start with a number!

Grouping Selectors

If you have elements with the same style definitions, like this:

```
h1 {
   text-align: center;
   color: red;
}

h2 {
   text-align: center;
   color: red;
}
```

```
p {
   text-align: center;
   color: red;
}
```

It will be better to group the selectors, to minimize the code. To group selectors, separate each selector with a comma. In the example below we have grouped the selectors from the code above:

```
h1, h2, p {
   text-align: center;
   color: red;
}
```

CSS Comments

Comments are used to explain the code, and may help when you edit the source code at a later date. Comments are ignored by browsers.

A CSS comment starts with /* and ends with */. Comments can also span multiple lines:

```
p {
   color: red;
   /* This is a single-line comment */
   text-align: center;
}
```

CSS Margin property is used to define the space around elements. It is completely transparent and doesn't have any background color. It clears an area around the element. Top, bottom, left and right margin can be changed independently using separate properties. You can also change all properties at once by using shorthand margin property.

The margin CSS shorthand property sets the margin area on all four sides of an element.

```
margin: 1em;
margin: 5% 0;
margin: 10px 50px 20px;
margin: 10px 50px 20px 0;
margin: 0;
```

Constituent properties

This property is a shorthand for the following CSS properties:

margin-bottom
margin-left
margin-right
margin-top

Syntax

```
/* Apply to all four sides */
margin: 1em;
margin: -3px;
```

```
/* vertical | horizontal */
margin: 5% auto;

/* top | horizontal | bottom */
margin: 1em auto 2em;

/* top | right | bottom | left */
margin: 2px 1em 0 auto;

/* Global values */
margin: inherit;
margin: initial;
margin: revert;
margin: unset;
```

The margin property may be specified using one, two, three, or four values. Each value is a <length>, a <percentage>, or the keyword auto. Negative values draw the element closer to its neighbors than it would be by default.

When one value is specified, it applies the same margin to all four sides. When two values are specified, the first margin applies to the top and bottom, the second to the left and right.

When three values are specified, the first margin applies to the top, the second to the right and left, the third to the bottom.When four values are specified, the margins apply to the top, right, bottom, and left in that order (clockwise).

Values

<length>

The size of the margin as a fixed value.

<percentage>

The size of the margin as a percentage, relative to the width of the containing block.

auto

The browser selects a suitable margin to use. For example, in certain cases this value can be used to center an element.

Description

This property can be used to set a margin on all four sides of an element. Margins create extra space around an element, unlike padding, which creates extra space within an element.

The top and bottom margins have no effect on non-replaced inline elements, such as or <code>.

Horizontal centering

To center something horizontally in modern browsers, you can use display: flex; justify-content: center;

However, in older browsers like IE8-9 that do not support Flexible Box Layout, these are not available. In order to center an element inside its parent, use margin: 0 auto;.

Margin collapsing

Elements' top and bottom margins are sometimes collapsed into a single margin that is equal to the larger of the two margins. See Mastering margin collapsing for more information.

Simple example

HTML

```
<div class="center">This element is centered.</div>

<div class="outside">This element is positioned outside of its container.</div>
```

Copy to Clipboard

CSS

```
.center {
  margin: auto;
  background: lime;
  width: 66%;
}
.outside {
  margin: 3rem 0 0 -3rem;
  background: cyan;
  width: 66%;
}
```

CSS for setting Padding value:

The *padding* property allows you to specify how much space should appear between the content of an element and its border −

The value of this attribute should be either a length, a percentage, or the word *inherit*. If the value is *inherit*, it will have the same padding as its parent element. If a percentage is used, the percentage is of the containing box.

The following CSS properties can be used to control lists. You can also set different values for the padding on each side of the box using the following properties −

- The **padding-bottom** specifies the bottom padding of an element.
- The **padding-top** specifies the top padding of an element.
- The **padding-left** specifies the left padding of an element.
- The **padding-right** specifies the right padding of an element.
- The **padding** serves as shorthand for the preceding properties.

Now, we will see how to use these properties with examples.

The padding-bottom Property

The *padding-bottom* property sets the bottom padding (space) of an element. This can take a value in terms of length of %.

Here is an example –

```
<html>
<head>
</head>

<body>
<p style = "padding-bottom: 15px; border:1px solid black;">
	This is a paragraph with a specified bottom padding
</p>

<p style = "padding-bottom: 5%; border:1px solid black;">
	This is another paragraph with a specified bottom padding in percent
</p>
</body>
</html>
```

The padding-top Property

The *padding-top* property sets the top padding (space) of an element. This can take a value in terms of length of %.

Here is an example

```
<html>
<head>
</head>
<body>
<p style = "padding-top: 15px; border:1px solid black;">
	This is a paragraph with a specified top padding
</p>

<p style = "padding-top: 5%; border:1px solid black;">

	This is another paragraph with a specified top padding in percent

</p>

</body>

</html>
```

The padding-left Property

The *padding-left* property sets the left padding (space) of an element. This can take a value in terms of length of %.

Here is an example –

```
<html>
<head>
</head>
<body>
```

```
<p style = "padding-left: 15px; border:1px solid black;">
	This is a paragraph with a specified left padding
</p>

<p style = "padding-left: 15%; border:1px solid black;">
	This is another paragraph with a specified left padding in percent
</p>
</body>
</html>
```

The padding-right Property

The *padding-right* property sets the right padding (space) of an element. This can take a value in terms of length of %.

Here is an example –

```
<html>
<head>
</head>
<body>
<p style = "padding-right: 15px; border:1px solid black;">
	This is a paragraph with a specified right padding
</p>
<p style = "padding-right: 5%; border:1px solid black;">
	This is another paragraph with a specified right padding in percent
</p>
</body>
</html>
```

The Padding Property

The *padding* property sets the left, right, top and bottom padding (space) of an element. This can take a value in terms of length of %.

Here is an example –

```
<html>
<head>
</head>
<body>
<p style = "padding: 15px; border:1px solid black;">
	all four padding will be 15px
</p>

<p style = "padding:10px 2%; border:1px solid black;">
	top and bottom padding will be 10px, left and right
	padding will be 2% of the total width of the document.
</p>

<p style = "padding: 10px 2% 10px; border:1px solid black;">
```

```
        top padding will be 10px, left and right padding will
        be 2% of the total width of the document, bottom padding will be 10px
    </p>

    <p style = "padding: 10px 2% 10px 10px; border:1px solid black;">
        top padding will be 10px, right padding will be 2% of
        the total width of the document, bottom padding and top padding will be 10px
    </p>
    </body>
    </html>
```

CSS Box model:

CSS box model is a container which contains multiple properties including borders, margin, padding and the content itself. It is used to create the design and layout of web pages. It can be used as a toolkit for customizing the layout of different elements. The web browser renders every element as a rectangular box according to the CSS box model.

Box-Model has multiple properties in CSS. Some of them are given below:

- borders
- margins
- padding
- Content

- ✓ **Border Area:** It is the area between the box's padding and margin. Its dimensions are given by the width and height of border.
- ✓ **Margin Area:** This area consists of space between border and margin. The dimensions of Margin area are the margin-box width and the margin-box height. It is useful to separate the element from its neighbors.
- ✓ **Padding Area:** It includes the element's padding. This area is actually the space around the content area and within the border box. Its dimensions are given by the width of the padding-box and the height of the padding-box.
- ✓ **Content Area:** This area consists of content like text, image, or other media content. It is bounded by the content edge and its dimensions are given by content box width and height.

Example:

```
<style>
        .main {
font-size:36px;
font-weight:bold;
text-align:center; color:red;
text-decoration:underline;
text-transform:lowercase;
 text-indent:80px;
letter-spacing:4px;
line-height:40px;
 direction: rtl;
 text-shadow:3px 1px blue;
 word-spacing:15px;
        }
        .gfg {
margin-left:60px;
border:50px solid #009900;
width:300px;    height:200px;
text-align:center;
```

```
padding:50px;
        }
        .gfg1 {
font-size:42px;
font-weight:bold;  color:#009900;
margin-top:60px;
background-color:#c5c5db;
        }
        .gfg2 {
font-size:18px;
font-weight:bold;
background-color:#c5c5db;
        }
 #box {
padding-top:40px;
width: 400px;    height: 100px;
border: 50px solid green;
margin: 50px;
text-align:center;
font-size:32px;
font-weight:bold;
      }
</style>
```

Text Decoration:

- text-decoration-line
- text-decoration-color
- text-decoration-style
- text-decoration-thickness
- text-decoration
- <!DOCTYPE html>
- <html>
- <head>
- <style>
- h1 {
- text-decoration: overline;
- }
- h2 {
- text-decoration: line-through;
- }
- h3 {
- text-decoration: underline;

- }
- p.ex {
- text-decoration: overline underline;
- }
- </style>
- </head>
- <body>
- <h1>Overline text decoration</h1>
- <h2>Line-through text decoration</h2>
- <h3>Underline text decoration</h3>
- Overline and underline text decoration.
- Note: It is not recommended to underline text that is not a link, as this often confuses
- the reader.
- </body>
- </html>

Text Transformation:

The text-transform property is used to specify uppercase and lowercase letters in a text.

It can be used to turn everything into uppercase or lowercase letters, or capitalize the first letter of each word:

```
p.uppercase {
  text-transform: uppercase;
}

p.lowercase {
  text-transform: lowercase;
}

p.capitalize {
  text-transform: capitalize;
}
```

- <!DOCTYPE html>
- <html>
- <head>
- <style>
- p.uppercase {

```
    text-transform: uppercase;
  }
  p.lowercase {
    text-transform: lowercase;
  }
  p.capitalize {
    text-transform: capitalize;
  }
  </style>
  </head  <body>
  <h1>Using the text-transform property</h1>
  <p class="uppercase">This text is transformed to uppercase.</p>
  <p class="lowercase">This text is transformed to lowercase.</p>
  <p class="capitalize">This text is capitalized.</p>
  </body>
  </html>
```

JavaScript:

JavaScript is a dynamically typed (also called loosely typed) scripting language. That is, in JavaScript variables can receive different data types over time. Datatypes are basically typed data that can be used and manipulated in a program.

The latest ECMAScript(ES6) standard defines following data types: Out of which six data types are Primitive(predefined).

- **Numbers**: Represent both integer and floating-point numbers. Example: 5, 6.5, 7 etc.
- **String**: A string is a sequence of characters. In JavaScript, strings can be enclosed within the single or double quotes. Example: “Hello GeeksforGeeks” etc.
- **Boolean**: Represent a logical entity and can have two values: true or false.
- **Null**: This type has only one value : *null.*
- **Undefined**: A variable that has not been assigned a value is *undefined.*
- **Symbol:** Unlike other primitive data types, it does not have any literal form. It is a built-in object whose constructor returns a symbol-that is unique.
- **bigint:** The bigint type represents the whole numbers that are larger than 2^{53}-1. To form a bigint literal number, you append the letter n at the end of the number.
- **Object**: It is the most important data-type and forms the building blocks for modern JavaScript. We will learn about these data types in detail in further articles.

Variables in JavaScript:

Variables in JavaScript are containers that hold reusable data. It is the basic unit of storage in a program.

- The value stored in a variable can be changed during program execution.

- A variable is only a name given to a memory location, all the operations done on the variable effects that memory location.
- In JavaScript, all the variables must be declared before they can be used.

Before ES2015, JavaScript variables were solely declared using the *var* keyword followed by the name of the variable and semi-colon. Below is the syntax to create variables in JavaScript:

var var_name;

var x;

The var_name is the name of the variable which should be defined by the user and should be unique. These types of names are also known as **identifiers**. The rules for creating an identifier in JavaScript are, the name of the identifier should not be any pre-defined word(known as keywords), the first character must be a letter, an underscore (_), or a dollar sign ($). Subsequent characters may be any letter or digit or an underscore or dollar sign.

Notice in the above code sample, we didn't assign any values to the variables. We are only saying they exist. If you were to look at the value of each variable in the above code sample, it would be *undefined.*

We can initialize the variables either at the time of declaration or also later when we want to use them. Below are some examples of declaring and initializing variables in JavaScript:

```
// declaring single variable
var name;
// declaring multiple variables
var name, title, num;

// initializing variables
var name = "Harsh";
name = "Rakesh";
let globalVar = "This is a global variable";
 function fun() {
  let localVar = "This is a local variable";
   console.log(globalVar);
  console.log(localVar);
}
 fun();
let globalVar = "This is a global variable";
 function fun() {
  let localVar = "This is a local variable";
}
 fun();
console.log(globalVar);
console.log(localVar);
```

A **computer program** is a list of "instructions" to be "executed" by a computer.In a programming language, these programming instructions are called **statements**.A **JavaScript program** is a list of programming **statements**.

JavaScript statements are composed of:Values, Operators, Expressions, Keywords, and Comments.This statement tells the browser to write "Hello Dolly." inside an HTML element with id="demo".Semicolons separate JavaScript statements.Add a semicolon at the end of each executable statement.

When separated by semicolons, multiple statements on one line are allowed. JavaScript ignores multiple spaces. You can add white space to your script to make it more readable.For best readability, programmers often like to avoid code lines longer than 80 characters.

If a JavaScript statement does not fit on one line, the best place to break it is after an operator. JavaScript statements can be grouped together in code blocks, inside curly brackets

The purpose of code blocks is to define statements to be executed together.One place you will find statements grouped together in blocks, is in JavaScript functions. JavaScript statements often start with a keyword to identify the JavaScript action to be performed.

Example:

```
var a, b, c;     // Declare 3 variables

a = 5;           // Assign the value 5 to a

b = 6;           // Assign the value 6 to b

c = a + b;       // Assign the sum of a and b to c
```

<u>Operators:</u>

Operators are used to identify the operation. Let us take a simple expression **4 + 5 is equal to 9**. Here 4 and 5 are called **operands** and '+' is called the **operator**. JavaScript supports the following types of operators.

- Arithmetic Operators
- Comparison Operators
- Logical (or Relational) Operators
- Assignment Operators
- Conditional (or ternary) Operators

Event handling in JavaScript. (6)

An event occurs when something happens in a browser window. The kinds of events that might occur are due to:

- A document loading
- The user clicking a mouse button
- The browser screen changing size

When a function is assigned to an event handler, that function is run when that event occurs.

A handler that is assigned from a script used the syntax '[element].[event] = [function];', where [element] is a page element, [event] is the name of the selected event and [function] is the name of the function that occurs when the event takes place.

For example:

```
document.onclick = clickHandler;
```

This handler will cause the function clickHandler() to be executed whenever the user clicks the mouse anywhere on the screen. Note that when an event handler is assigned, the function name does not end with parentheses. We are just pointing the event to the name of the function. The clickHandler() function is defined like this:

```
function clickHandler(event) {  //some code here }
```

By convention the event is represented by the variable 'event()'. In some browsers the event must be explicitly passed to the function, so as a precaution it's often best to include a conditional to test that the event() variable has been passed, and if it hasn't then to use an alternative method that works on those other browsers:

```
function clickHandler(event) {
        event = event || window.event;
         //some code here
         }
```

Elements within a document can also be assigned event handlers. For example:

```
document.getElementsByTagName('a')[0].onclick = linkHandler;
```

This will cause the linkHandler() function to be executed when the user clicks the first link on the page.

Keep in mind that this style of handler assignment depends on the link's position inside the page. If another link tag is added before this one, it will take over the handler from the original link. A best practice is to maintain the separation of code and page structure by assigning each link an identifier by using the id attribute.

```
<a id="faqLink" href="faq.html">Faq</a>
```

A handler assignment can then work regardless of where the element is positioned.

```
document.getElementById('faqLink').onclick = linkHandler;
```

Handlers are divided into two types:

i. Interactive event handlers - depend on user interaction with the HTML page;
   ex. clicking a button
ii. Non-Interactive event handlers - do not need user interaction;
   ex. on load

Event Attributes

Below is the event attributes that can be inserted into different HTML elements to define event actions. IE: Internet Explorer, F: Firefox, O: Opera, W3C: W3C Standard.

Attribute	The event occurs when...	Attribute	The event occurs when...
`Onblur`	An element loses focus	`Onload`	A page or image hasfinished loading
`Onchange`	The content of a field changes	`onmousedown`	A mouse button is pressed

Onclick	Mouse clicks an object	onmousemove	The mouse is moved
ondblclick	Mouse double-clicks an object	Onmouseout	The mouse is movedoff an element
Onerror	An error occurs when loading a document or an image	Onmouseover	The mouse is movedover an element
Onfocus	An element gets focus	Onmouseup	A mouse button is released
Onkeydown	A keyboard key is pressed	Onresize	A window or frame is resized
onkeypress	A keyboard key is pressed or held down	Onselect	Text is selected
Onkeyup	A keyboard key is released	Onunload	The user exits the page

EXAMPLE:

```
<script>
function armschk()
{
// program to check an Armstrong number of n digits
// take an input
const number = prompt("Enter a positive integer");
const numberOfDigits = number.length;
let sum = 0;

// create a temporary variable
let temp = number;
while (temp > 0) {
   let remainder = temp % 10;
   sum += remainder ** numberOfDigits;
   // removing last digit from the number
   temp = parseInt(temp / 10); // convert float into integer
}
if (sum == number) {
   console.log(`${number} is an Armstrong number`);
}
else {
   console.log(`${number} is not an Armstrong number.`);
}
}
</script>
```

UNIT III SERVER SIDE SCRIPTING

JavaBeans Classes in JSP:

Bean means component. Components mean reusable objects. JavaBean is a reusable component. Java Bean is a normal java class that may declare properties, setter, and getter methods in order to represent a particular user form on the server-side. JavaBean is a java class that is developed with a set of conventions. The conventions are:

- The class should be public and must not be final, abstract.
- Recommend to implement serializable.
- Must have direct/indirect param constructor.
- Member variables should be non-static and private.
- Every bean property should have one setter method and one getter method with the public modifier.

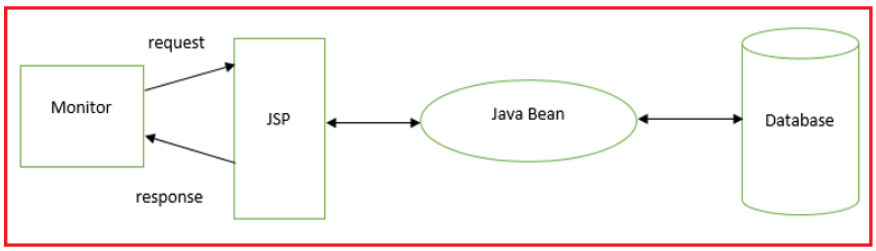
Java Bean is useful to combine multiple simple values to an object and to send that object from one resource to another resource or one layer to another layer. For example, to keep form data into a single object we can use java bean. Java Bean is a specially constructed Java class written in Java and coded according to the Java Beans API specification.

Why do we need to use JavaBeans in JSP?

1. It provides a default, no-argument constructor.
2. It should be serializable and implement the Serializable interface.
3. It may have a number of properties/variables which can be read or written.
4. It may have a number of "getter" and "setter" methods for the properties.

Working of JavaBeans in JSP

First, the browser sends the request for the JSP page. Then the JSP page accesses Java Bean and invokes the business logic. After invoking the business logic Java Bean connects to the database and gets/saves the data. At last, the response is sent to the browser which is generated by the JSP.



Advantages of Java Beans

1. It is easy to reuse the software components.
2. The properties and methods of Java Bean can be exposed to another application.

Disadvantages of Java Beans

1. JavaBeans are mutable objects so it cannot take advantage of immutable objects.
2. JavaBeans will be in an inconsistent state because of creating the setter and getter method for each property separately.

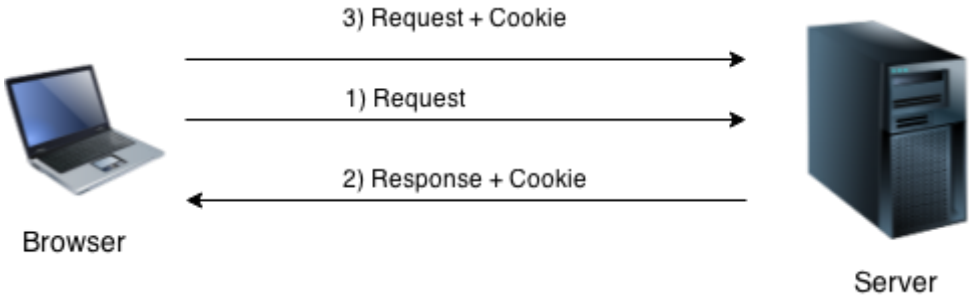
Java Bean Properties

1. **getPropertyName():** This method is used to read the property which is also called accessor.
2. **setPropertyName():** This method is used to write the property which is also called mutator.

Use of Cookies class in Servlet:

By default, each request is considered as a new request. In cookies technique, we add cookie with response from the servlet. So cookie is stored in the cache of the browser. After that if

request is sent by the user, cookie is added with request by default. Thus, we recognize the user as the old user.




```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class FirstServlet extends HttpServlet {

  public void doPost(HttpServletRequest request,
HttpServletResponse response){
    try{

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    String n=request.getParameter("userName");
    out.print("Welcome "+n);

    Cookie ck=new Cookie("uname",n);//creating cookie object
    response.addCookie(ck);//adding cookie in the response

    //creating submit button
    out.print("<form action='servlet2'>");
    out.print("<input type='submit' value='go'>");
    out.print("</form>");

    out.close();
```

```
}catch(Exception e){System.out.println(e);}
    }
}
```

Types of drivers in JDBC

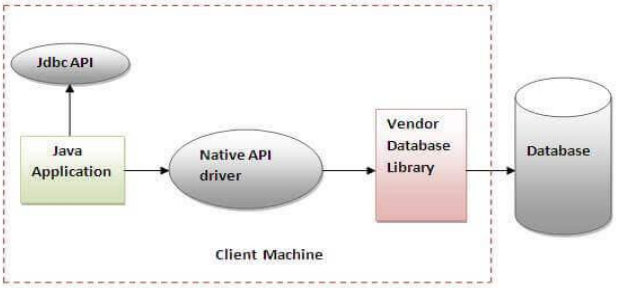


Figure- Native API Driver

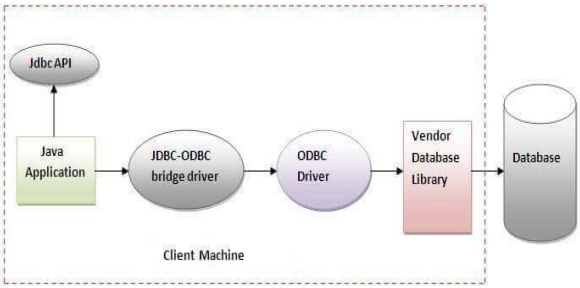


Figure- JDBC-ODBC Bridge Driver

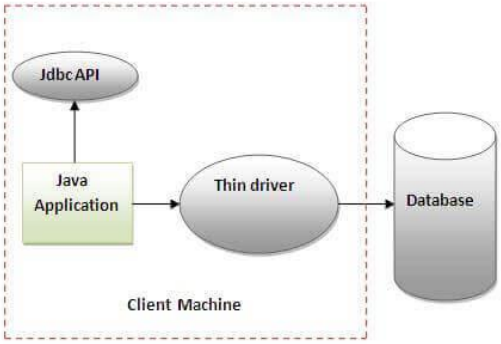


Figure- Thin Driver

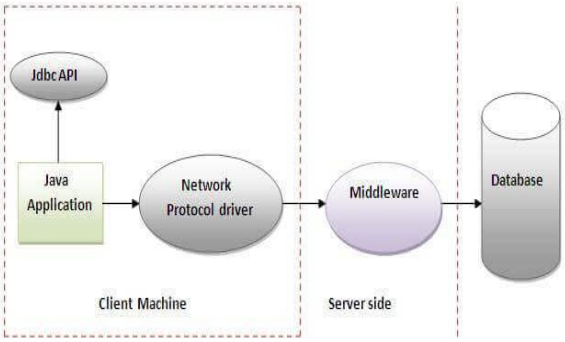
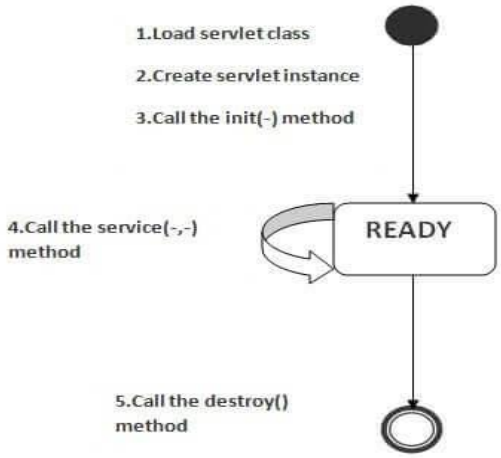


Figure- Network Protocol Driver

Life cycle of servlet:

1. Servlet class is loaded.
2. Servlet instance is created.
3. init method is invoked.
4. service method is invoked.
5. destroy method is invoked.



As displayed in the above diagram, there are three states of a servlet: new, ready and end. The servlet is in new state if servlet instance is created. After invoking the init() method, Servlet comes in the ready state. In the ready state, servlet performs all the tasks. When the web container invokes the destroy() method, it shifts to the end state.

Example:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"

  "http://www.w3.org/TR/html4/loose.dtd">

<html>    <head>

      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">

      <title>JSP Page</title>

      <script>

         function checkNumber(f)

         {

            var a = window.event.keycode;

            if(!(a>=48 && a<=57))

               alert("Type a number between 0 and 9")

         }

         </script>    </head>

    <body><br><br><center>

      <form action=" Interest_Calculation.js"><pre>

<h1>Principal :: <input type=text name=principle value=0
                              onkeypress="checkNumber(this.form)" ><br>
```

```
   No. of Years :: <input type=text name=year value=0
                        onkeypress="checkNumber(this.form)" ><br>

   Rate of Interest :: <input type=text name=interest value=0
                        onkeypress="checkNumber(this.form)" ><br>

   <br>

<input type=submit value="Submit"></h1>

</pre> </form></center>

</body>   </html>
```

Interest_Calculation.jsp

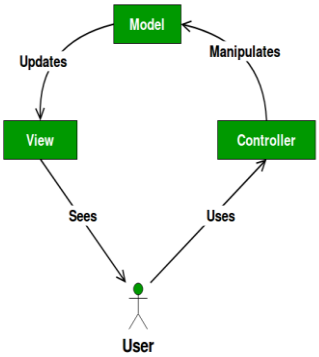
```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"

   pageEncoding="ISO-8859-1"%>

<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<html>

<head>

<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">

<title>Guru Success Page</title>

</head>

<body>

<%

      float p=Float.parseFloat(request.getParameter("p"));

      float r=Float.parseFloat(request.getParameter("r"));

      float n=Float.parseFloat(request.getParameter("n"));


      float i=(p*r*n)/100;

      out.println("<br><h1>Simple Interest will be : "+i);


%>

</body>

</html>
```

Model-View-Controller Paradigm:

The Model View Controller (MVC) design pattern specifies that an application consist of a data model, presentation information, and control information. The pattern requires that each of these be separated into different objects.

MVC is more of an architectural pattern, but not for complete application. MVC mostly relates to the UI / interaction layer of an application. You're still going to need business logic layer, maybe some service layer and data access layer.

UML Diagram MVC Design Pattern



ii. Tag Libraries and Files

Standard JavaServer Pages technology allows vendors to create custom JSP tag libraries.

A tag library defines a collection of custom actions. The tags can be used directly by developers in manually coding a JSP page, or automatically by Java development tools. A tag library must be portable between different JSP container implementations.

For information beyond what is provided here regarding tag libraries and the standard JavaServer Pages tag library framework, refer to the following resources.

- Sun Microsystems *JavaServer Pages Specification, Version 1.1*
- Sun Microsystems Javadoc for the javax.servlet.jsp.tagext package, at the following Web site:
- http://java.sun.com/j2ee/j2sdkee/techdocs/api/javax/servlet/jsp/tagext/package-summary.html

Overview of a Custom Tag Library Implementation

A custom tag library is made accessible to a JSP page through a taglib directive of the following general form:

<%@ taglib uri="*URI*" prefix="*prefix*" %>

Note the following:

- The tags of a library are defined in a *tag library description* file, as described in "Tag Library Description Files".
- The URI in the taglib directive specifies where to find the tag library description file, as described in "The taglib Directive". It is possible to use *URI shortcuts*, as described in "Use of web.xml for Tag Libraries".

- The prefix in the taglib directive is a string of your choosing that you use in your JSP page with any tag from the library.

Assume the taglib directive specifies a prefix oracust:

<%@ taglib uri="*URI*" prefix="oracust" %>

session tracking techniques with suitable example.

HTTP is a "stateless" protocol which means each time a client retrieves a Web page, the client opens a separate connection to the Web server and the server automatically does not keep any record of previous client request. Still there are following three ways to maintain session between web client and web server.

Cookies

A webserver can assign a unique session ID as a cookie to each web client and for subsequent requests from the client they can be recognized using the received cookie. This may not be an effective way because many time browser does not support a cookie, so I would not recommend to use this procedure to maintain the sessions.

Hidden Form Fields

A web server can send a hidden HTML form field along with a unique session ID as follows –

<input type = "hidden" name = "sessionid" value = "12345">

This entry means that, when the form is submitted, the specified name and value are automatically included in the GET or POST data. Each time when web browser sends request back, then session_id value can be used to keep the track of different web browsers. This could be an effective way of keeping track of the session but clicking on a regular (<A HREF...>) hypertext link does not result in a form submission, so hidden form fields also cannot support general session tracking.

URL Rewriting

You can append some extra data on the end of each URL that identifies the session, and the server can associate that session identifier with data it has stored about that session. For example, with http://tutorialspoint.com/file.htm;sessionid = 12345, the session identifier is attached as sessionid = 12345 which can be accessed at the web server to identify the client. URL rewriting is a better way to maintain sessions and it works even when browsers don't support cookies. The drawback of URL re-writing is that you would have to generate every URL dynamically to assign a session ID, even in case of a simple static HTML page.

The HttpSession Object

Apart from the above mentioned three ways, servlet provides HttpSession Interface which provides a way to identify a user across more than one page request or visit to a Web site and to store information about that user. The servlet container uses this interface to create a session between an HTTP client and an HTTP server. The session persists for a specified time period, across more than one connection or page request from the user. You would get HttpSession object by calling the public method getSession()of HttpServletRequest, as below −

HttpSession session = request.getSession();

You need to call *request.getSession()* before you send any document content to the client.
Here is a summary of the important methods available through HttpSession object

```
import java.io.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class DateServlet extends HttpServlet {
public void doGet(HttpServletRequest request,HttpServletResponse response)
                                                    throws ServletException, IOException {
HttpSession hs = request.getSession(true);
response.setContentType("text/html");
PrintWriter pw = response.getWriter();
pw.print("<B>");
Date date = (Date)hs.getAttribute("date");
if(date != null) {       pw.print("Last access: " + date + "<br>");       }
date = new Date();
hs.setAttribute("date", date);
pw.println("Current date: " + date);
                    }}
```

Example:

```
<html>
<head>
<script>
function Form_Validate ()
{
   var name = document.forms["RegForm"]["Name"];
   var email = document.forms["RegForm"]["EMail"];
   var phone = document.forms["RegForm"]["Telephone"];
   var what =  document.forms["RegForm"]["Subject"];
   var password = document.forms["RegForm"]["Password"];
   var address = document.forms["RegForm"]["Address"];

   if (name.value == "")
```

```
    {
        window.alert("Please enter your name.");
        name.focus();
        return false;
    }
    if (address.value == "")
    {
        window.alert("Please enter your address.");
        address.focus();
        return false;
    }
      if (phone.value == "")
    {
        window.alert("Please enter your telephone number.");
        phone.focus();    return false;     }
    if (password.value == "")
    {
        window.alert("Please enter your password");
        password.focus();
        return false;
    }
    if (what.selectedIndex < 1)
    {
        alert("Please enter your course.");
        what.focus();    return false;     }
      return true;    }</script>

<title> SCHOOL MANAGEMENT SYSTEM </title>
</head>
<body>
<div>
```

```
<h1 style="text-align: center"> SCHOOL MANAGEMENT SYSTEM </h1>

<form name="RegForm" action="/submit.php" onsubmit="return Form_Validate()"
                                                                    method="post">

   <p>Name <input type="text" size=65 name="Name"> </p>

   <p> Address <input type="text" size=65 name="Address"> </p>

   <p>E-mail Address <input type="text" size=65 name="EMail">  </p>

   <p>Password <input type="text" size=65 name="Password"> </p>

   <p>Telephone <input type="text" size=65 name="Telephone"> </p>

   <p>Sex <input type="radio" name="msex" value="Male" /> <span>Male</span>

            <input type="radio" name="fsex" value="Female" />
<span>Female</span></p><br>

   <p>Language <input type="checkbox" name="en" value="en" checked />
<span>English</span>

            <input type="checkbox" name="nonen" value="noen" /> <span>Non
English</span></p><br>


   <p>Country    : <select name="country">

                  <option selected="" value="Default">(Please select a country)

                    </option>

                  <option value="AF">Australia</option>

                  <option value="AL">Canada</option>

                  <option value="DZ">India</option>

                  <option value="AS">Russia</option>

                  <option value="AD">USA</option>

                  </select>  </p><br>

   <p>SELECT YOUR COURSE

      <select type="text" value="" name="Subject">

                  <option>BTECH</option>

                  <option>BE</option>

                  <option>M.E</option>

                  <option>M.TECH</option>

                  <option>MBA</option>

                </select></p><br>
```

```
<p>Comments:  <textarea cols="55" name="Comment">  </textarea></p><br>
<p><input type="submit" value="Submit" name="Submit">
      <input type="reset" value="Reset" name="Reset">
</p>
</form>
</div></body> </html>
```

UNIT IV DOM AND XML

Intrinsic Event Handling

An intrinsic event determines when an associated script will run. However, not every intrinsic event works with every (X)HTML element. This table illustrates which events and tags work together.

Event	Works With	When
onblur	a, area, button, input, label, select, textarea	the visitor leaves an element that was previously in focus (see onfocus below)
onchange	input, select, textarea	the visitor modifies the value or contents of the element
onclick	All elements except applet, base, basefont, br, font, frame, frameset, head, html, iframe, meta, param, script, style, title	the visitor clicks on the specified area ...

DOM for Modifying Element Style

The Document Object Model (DOM) is a **programming interface** for **HTML** and **XML**(Extensible markup language) documents. It defines the **logical structure** of documents and the way a document is accessed and manipulated.
Note: It is called as a Logical structure because DOM doesn't specify any relationship between objects.
DOM is a way to represent the webpage in the structured hierarchical way so that it will become easier for programmers and users to glide through the document. With DOM, we can easily access and manipulate tags, IDs, classes, Attributes or Elements using commands or methods provided by Document object.

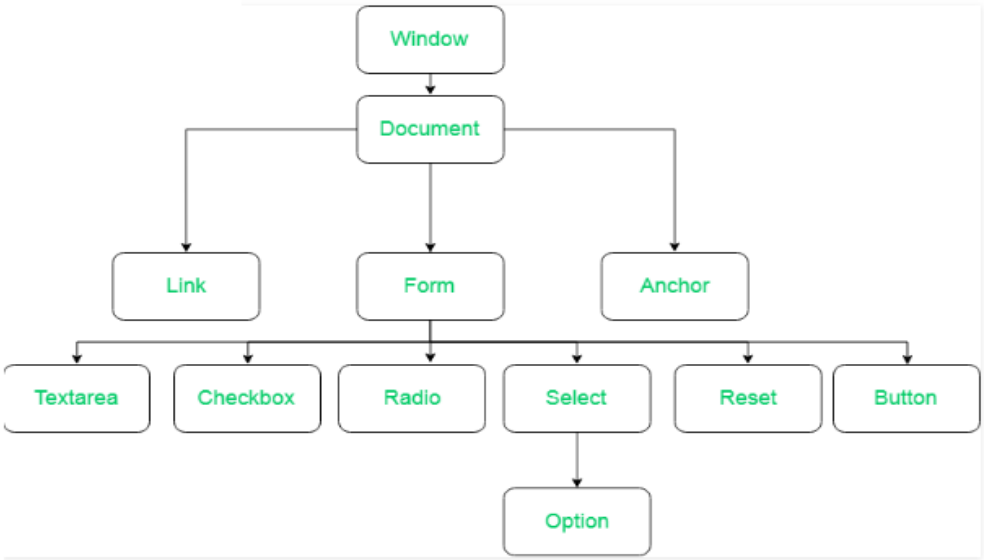
Structure of DOM:
DOM can be thought of as Tree or Forest(more than one tree). The term **structure model** is sometimes used to describe the tree-like representation of a document. One important property of DOM structure models is ***structural isomorphism***: if any two DOM implementations are used to create a representation of the same document, they will create the same structure model, with precisely the same objects and relationships.

Documents are modeled using objects, and the model includes not only the structure of a document but also the behavior of a document and the objects of which it is composed of like tag elements with attributes in HTML.

Properties of DOM:
Let's see the properties of document object that can be accessed and modified by the document object.

1. **Window Object:** Window Object is at always at top of hierarchy.
2. **Document object:** When HTML document is loaded into a window, it becomes a document object.
3. **Form Object:** It is represented by *form* tags.
4. **Link Objects:** It is represented by *link* tags.
5. **Anchor Objects:** It is represented by *a href* tags.

6. **Form Control Elements:**: Form can have many control elements such as text fields, buttons, radio buttons, and checkboxes, etc.



Methods **of Document Object**:

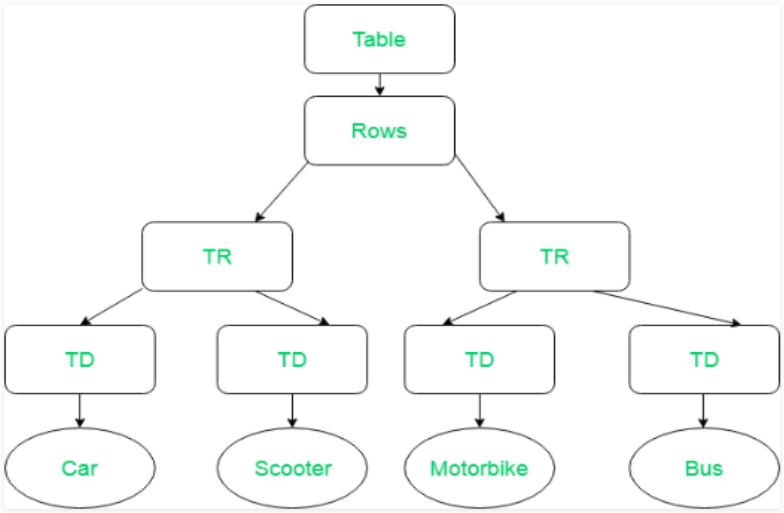
1. **write("string"):** writes the given string on the document.
2. **getElementById():** returns the element having the given id value.
3. **getElementsByName():** returns all the elements having the given name value.
4. **getElementsByTagName():** returns all the elements having the given tag name.
5. **getElementsByClassName():** returns all the elements having the given class name.

Example:

```
<Table>

  <ROWS>

    <TR>

      <TD>Car</TD>

      <TD>Scooter</TD>

    </TR>

    <TR>

      <TD>MotorBike</TD>

      <TD>Bus</TD>

    </TR>

  </ROWS>

</Table>
```

1. The Document Object Model is not a **binary description** where it does not define any binary source code in its interfaces.

2. The Document Object Model is not used to describe **objects in** *XML* **or** *HTML*whereas the DOM describes XML and HTML documents as objects.
3. The Document Object Model is not represented by **set of data structures**; it is an interface which specifies object representation.
4. The Document Object Model does not show **criticality of objects** in documents i.e it doesn't have information about which object in document is appropriate to the context and which is not.



Levels **of DOM**:

1. *Level 0:* Provides low-level set of interfaces.
2. *Level 1:* DOM level 1 can be described in two parts: **CORE** and **HTML**.
   - ***CORE*** provides a low-level interfaces that can be used to represent any structured document.
   - ***HTML*** provides high-level interfaces that can be used to represent HTML document.
3. *Level 2: consists of six specifications: CORE2, VIEWS, EVENTS, STYLE, TRAVERSAL and RANGE.*
   - ***CORE2***: extends functionality of CORE specified by DOM level 1.
   - ***VIEWS***: views allows programs to dynamically access and manipulate content of document.
   - ***EVENTS***: Events are scripts that is either executed by browser when user reacts to web page.
   - ***STYLE***: allows programs to dynamically access and manipulate content of style sheets.
   - ***TRAVERSAL***: allows programs to dynamically traverse the document.
   - ***RANGE***: allows programs to dynamically identify a range of content in document.
4. *Level 3: consists of five different specifications: CORE3, LOAD and SAVE, VALIDATION, EVENTS, and XPATH.*
   - *CORE3*: extends functionality of CORE specified by DOM level 2.
   - *LOAD and SAVE*: allows program to dynamically load the content of XML document into DOM document and save the DOM Document into XML document by serialization.
   - *VALIDATION*: allows program to dynamically update the content and structure of document while ensuring document remains valid.
   - *EVENTS*: extends functionality of Events specified by DOM Level 2.
   - *XPATH*: XPATH is a path language that can be used to access DOM tree.

DOM Parser:

```
import java.io.*;
import org.w3c.dom.*;
import javax.xml.parsers.*;
public class SimpleWalker
{
protected DocumentBuilder docBuilder;
protected Element root;
public SimpleWalker() throws Exception
{
DocumentBuilderFactory dbf=DocumentBuilderFactory.newInstance();
docBuilder=dbf.newDocumentBuilder();
DOMImplmention domImp=docBuilder.getDOMImplementation();
if(domImp.hasFeature("XML","2.0"))
{
System.out.println("Parser supports extended interfaces");
}
}
public void parse(String fileName) throws Exception
{
Document doc=docBuilder.parse(new FileInputStream(fileName));
root=doc.getDocumentElement();
System.out.println("Root element is"+ root.getNodeName());
}
public void printAllElements() throws Exception
{
printElement("",root);
}
public void printElement(String indent, Node aNode)
{
System.out.println(indent+"<"+aNode.getNodeName()+">");
Node child=aNode.getFirstChild();
```

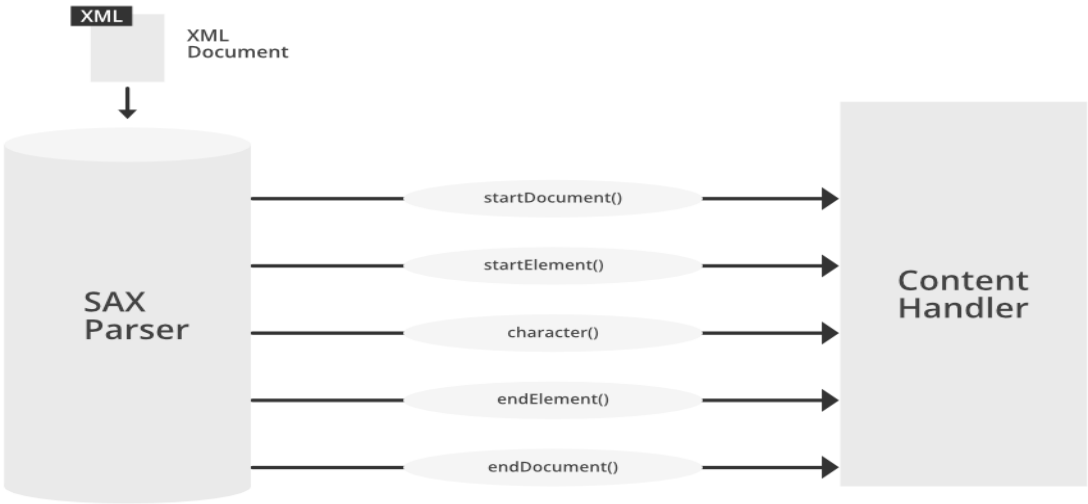
```
while(child!=null){
printElement(indent+"\t",child);
child=child.getNextSibling();
}
System.out.println(indent+"</"+aNode.getNodeName()+">");
}
public static void main(String args[]) throws Exception
{
SimpleWalker sw=new SimpleWalker();
sw.parse(args[0]);
sw.printAllElements();
}
}
```

Parsers are used to process XML documents. The parser examines the XML document, checks for errors, and then validate it against a schema or DTD if it's a validating parser. The next step is determined by the parser in use. It may copy the data into a data structure native to the computer language you're using on occasion. It may also apply styling to the data or convert it into a presentation format.

Apart from triggering certain events, the SAX parser does nothing with the data. It is up to the SAX parser's user to decide.

1. XML Text Nodes
2. XML Element Starts and Ends
3. XML Processing Instructions
4. XML Comments

Properties of SAX Parser is depicted below as follows:



Note: WHEN SAX is better than DOM

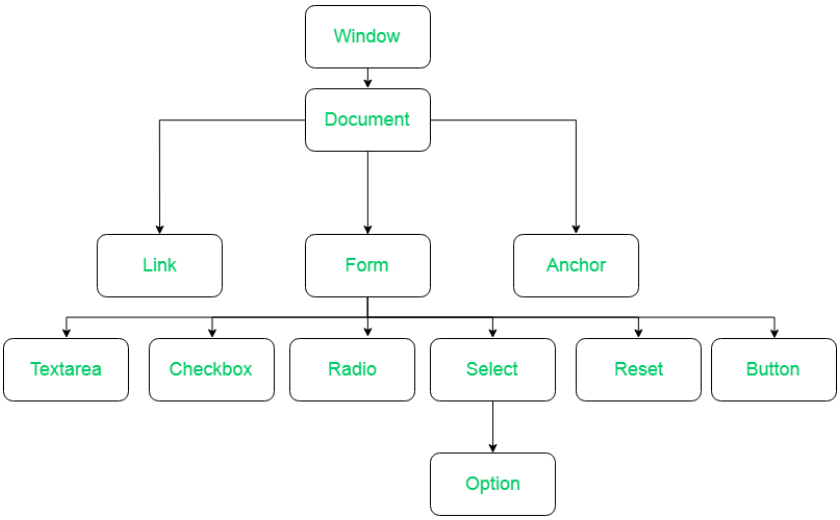
- It parses the XNIL file as a stream rather than allocating RAM for the complete file.
- Since, it uses less memory and is faster than the DOM Parser because the complete file is not stored in memory.
- Therefore, it is considered to be useful in parsing large XML files.

Tip: Drawbacks of SAX over DOM

- It parses the XNIL file as a stream rather than allocating RAM for the complete file.
- There are no update methods in the SAX Parser. Since the complete file isn't kept in memory, it is possible to access items only in a sequential manner and the elements cannot be accessed randomly.

ii. DOM History

The Document Object Model (DOM) is a ***programming interface*** for **HTML(HyperText Markup Language)** and **XML**(Extensible markup language) documents. It defines the **logical structure** of documents and the way a document is accessed and manipulated.
It is called a Logical structure because DOM doesn't specify any relationship between objects.
DOM is a way to represent the webpage in a structured hierarchical way so that it will become easier for programmers and users to glide through the document. With DOM, we can easily access and manipulate tags, IDs, classes, Attributes, or Elements of HTML using commands or methods provided by the Document object. Using DOM, the JavaScript gets access to HTML as well as CSS of the web page and can also add behavior to the HTML elements. so basically **Document Object Model is an API that represents and interacts with HTML or XML documents.**
Why DOM is required?
HTML is used to **structure** the web pages and Javascript is used to add **behavior** to our web pages. When an HTML file is loaded into the browser, the javascript can not understand the HTML document directly. So, a corresponding document is created(DOM). **DOM is basically the representation of the same HTML document but in a different format with the use of objects**. Javascript interprets DOM easily i.e javascript can not understand the tags(<h1>H</h1>) in HTML document but can understand object h1 in DOM. Now, Javascript can access each of the objects (h1, p, etc) by using different functions.
Structure of DOM: DOM can be thought of as a Tree or Forest(more than one tree). The term **structure model** is sometimes used to describe the tree-like representation of a document. Each branch of the tree ends in a node, and each node contains objects Event listeners can be added to nodes and triggered on an occurrence of a given event. One important property of DOM structure models is ***structural isomorphism***: if any two DOM implementations are used to create a representation of the same document, they will create the same structure model, with precisely the same objects and relationships.
Documents are modeled using objects, and the model includes not only the structure of a document but also the behavior of a document and the objects of which it is composed like tag elements with attributes in HTML.
Properties of DOM: Let's see the properties of the document object that can be accessed and modified by the document object.



Representation of the DOM

- **Window Object:** Window Object is object of the browser which is always at top of the hierarchy. It is like an API that is used to set and access all the properties and methods of the browser. It is automatically created by the browser.
- **Document object:** When an HTML document is loaded into a window, it becomes a document object. The 'document' object has various properties that refer to other objects which allow access to and modification of the content of the web page. If there is a need to access any element in an HTML page, we always start with accessing the 'document' object. Document object is property of window object.
- **Form Object:** It is represented by ***form*** tags.
- **Link Object:** It is represented by ***link*** tags.
- **Anchor Object:** It is represented by ***a href*** tags.
- **Form Control Elements:**: Form can have many control elements such as text fields, buttons, radio buttons, checkboxes, etc.

Methods of Document Object:

- **write("string"):** Writes the given string on the document.
- **getElementById():** returns the element having the given id value.
- **getElementsByName():** returns all the elements having the given name value.
- **getElementsByTagName():** returns all the elements having the given tag name.

getElementsByClassName(): returns all the elements having the given class name

Transforming XML Documents data to HTML page.

When building an XSLT solution that uses Internet Explorer, you might want to use data islands as a convenient way to access XML data from an HTML page. Because XSLT is itself an XML grammar, data islands can be used to load style sheets as well. The simple-islands.htm example in XSLT Sample: Breakfast Menu uses two data islands to load both the source XML data and the XSLT style sheet. The transformNode method is used on these two data islands to generate HTML code that can then be inserted into the page.

When the simple-islands.htm page, including the data islands, is completely loaded, the onload event fires and the transformNode method is executed on the XML source data. The result of this call is XML text, which in this example is well-formed HTML text. You can feed this text directly into the <xslTarget> element as in xslTarget.innerHTML = source.transformNode(style.XMLDocument)

The object returned from a data island is a DHTML DOM object (in this case it is the DHTML object associated with the <XML> element), not an XML DOM object. The

style.XMLDocument property ensures that a style object is passed to the transformNode method as an XML DOM object.

There is no need to write source.XMLDocument.transformNode(style.XMLDocument), because transformNode is an XML DOM method, not a DHTML method. When an XML DOM method is called on a DHTML object corresponding to the XML data island element, the script engine automatically resolves it to the XML DOM Document node. In general, the XMLDocument property is not required if another DOM property or method is being applied

Selecting XML Data by using XPATH.

- XPath is a syntax for defining parts of an XML document
- XPath uses path expressions to navigate in XML documents
- XPath contains a library of standard functions
- XPath is a major element in XSLT and in XQuery
- XPath is a W3C recommendation

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
<book category="cooking">
  <title lang="en">Everyday Italian</title>
  <author>Giada De Laurentiis</author>
  <year>2005</year>
  <price>30.00</price>
</book>

<book category="children">
  <title lang="en">Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>

<book category="web">
  <title lang="en">XQuery Kick Start</title>
  <author>James McGovern</author>
  <author>Per Bothner</author>
  <author>Kurt Cagle</author>
  <author>James Linn</author>
  <author>Vaidyanathan Nagarajan</author>
  <year>2003</year>
  <price>49.99</price>
</book>

<book category="web">
  <title lang="en">Learning XML</title>
  <author>Erik T. Ray</author>
  <year>2003</year>
  <price>39.95</price>
</book>
</bookstore>
```

XSLT and write XSLT for displaying employee details from XML Document into Browser.

- Multiple XML developers may choose the same element & attribute names for their standards

Each of them, may mean entirely different things.

For a processing application, to understand & associate the correct elements,

It must know which tag set the elements come from.

- XML Namespaces provide a method to avoid element name conflicts.
- Namespaces use a colon-delimited prefix

To associate external semantics with elements

– That can be identified via a Universal Resource Identifier (URI).

<table> <tr> <td>Apples</td> <td>Bananas</td> </tr> </table>	<table> <name>African Coffee Table</name> <width>80</width> <length>120</length> </table>
<h:table> <h:tr> <h:td>Apples</h:td> <h:td>Bananas</h:td> </h:tr> </h:table>	<f:table> <f:name>African Coffee Table</f:name> <f:width>80</f:width> <f:length>120</f:length> </f:table>

```
<?xml version="1.0"?>

<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">

  <xsl:template match="/book">

    <html>

     <body>

      <b>Title: </b> <xsl:value-of select="title" />

      <p/>

      <b>By: </b> <xsl:value-of select="author" />

      <p/>

      <b>Cost: </b> <xsl:value-of select="price" />

      <p/>

      <b>Category: </b> <xsl:value-of select="category" />

      <p/>
```

```
    <b>Description</b>
    <p/>
    <i><xsl:value-of select="summary" /></i>
   </body>
  </html>
 </xsl:template>
</xsl:stylesheet>
```

XML DTD:

An XML document with correct syntax is called "Well Formed".

An XML document validated against a DTD is both "Well Formed" and "Valid".

DTD stands for Document Type Definition.

A DTD defines the structure and the legal elements and attributes of an XML document.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note SYSTEM "Note.dtd">
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
</note>
```

```
<!DOCTYPE note
[
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
]>
```

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE note [
<!ENTITY nbsp " ">
<!ENTITY writer "Writer: Donald Duck.">
<!ENTITY copyright "Copyright: W3Schools.">
]>

<note>
<to>Tove</to>
<from>Jani</from>
```

<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
<footer>&writer; &copyright;</footer>
</note>

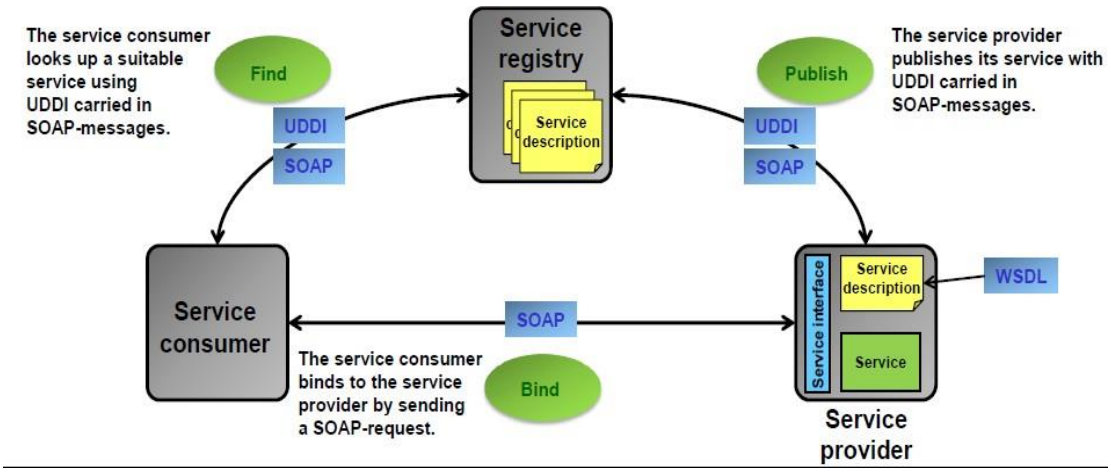
XPATH used in XSLT:

- – XPath is a syntax for defining parts of an XML document
- – XPath uses path expressions to navigate in XML documents
- – XPath contains a library of standard functions
- – XPath is a major element in XSLT
- – XPath is also used in XQuery, XPointer and XLink
- – XPath is a W3C recommendation
    - XPath was created
- – To reduce the amount of time to find the elements and attributes desired by an author

- XPath uses path expressions
    - – To select nodes or node-sets in an XML document.
    - – XPath expressions can be used in
    - – JavaScript,
    - – Java,
    - – XML Schema,
    - – PHP,
    - – Python,
    - – C and C++, and lots of other languages.
- XPath includes over 100 built-in functions.
- There are functions for
    - – String values, Numeric values,
    - – Date and time comparison,
    - – Node and QName manipulation,
    - – Sequence manipulation,
    - – Boolean values, and more.
- XPath was designed to be used by
    - – XSLT,
    - – XPointer and
    - – Other XML parsing software.
    - – Without XPath knowledge
    - – You will not be able to create XSLT documents.

UNIT V AJAX AND WEB SERVICES

Architecture of web service



The combo SOAP+WSDL+UDDI defines a general model for a web service architecture.SOAP: Simple Object Access Protocol
WSDL: Web Service Description Language
UDDI: Universal Description and
Discovery ProtocolService consumer:
User of a service
Service provider: Entity that implements a service (=server)

WSDL and its elements.

WSDL breaks down web services into three specific, identifiable elements that can be combined or reused once defined.

The three major elements of WSDL that can be defined separately are

- ✓ Types
- ✓ Operations
- ✓ Binding

A WSDL document has various elements, but they are contained within these three main elements, which can be developed as separate documents and then they can be combined or reused to form complete WSDL files.

WSDL Elements

A WSDL document contains the following elements

Definition − It is the root element of all WSDL documents. It defines the name of the web service, declares multiple namespaces used throughout the remainder of the document, and contains all the service elements described here.

Data types − The data types to be used in the messages are in the form of XML schemas.

Message − It is an abstract definition of the data, in the form of a message presented either as an entire document or as arguments to be mapped to a method invocation.

Operation − It is the abstract definition of the operation for a message, such as naming a method, message queue, or business process, that will accept and process the message.

Port type − It is an abstract set of operations mapped to one or more end-points, defining the collection of operations for a binding; the collection of operations, as it is abstract, can be mapped to multiple transports through various bindings.

Binding − It is the concrete protocol and data formats for the operations and messages defined for a particular port type.

Port − It is a combination of a binding and a network address, providing the target address of the service communication.

Service − It is a collection of related end-points encompassing the service definitions in the file; the services map the binding to the port and include any extensibility definitions.

In addition to these major elements, the WSDL specification also defines the following utility elements
Documentation − This element is used to provide human-readable documentation and can be included inside any other WSDL element.
Import − This element is used to import other WSDL documents or XML Schemas.
NOTE − WSDL parts are usually generated automatically using web services-aware tools.

The WSDL Document Structure
The main structure of a WSDL document looks like this −

```
<definitions>
  <types>
    definition of types
  </types>
  <message>
    definition of a message
  </message>
  <portType>
    <operation>
      definition of a operation
    </operation>
  </portType>
  <binding>
    definition of a binding
  </binding>
  <service>
    definition of a service
  </service>
</definitions>
```

A WSDL document can also contain other elements, like extension elements and a service element that makes it possible to group together the definitions of several web services in one single WSDL document.

A Web service is a software system designed to support interoperable machine-to machine interaction over a network. It has an interface described in a machine-processable format (specifically WSDL). Other systems interact with the Web service in a manner prescribed by its description using SOAP-messages, typically conveyed using HTTP with an XML serialization in conjunction with other Web-related standards."

The key elements are:

- Machine-to-machine communication.
- Machine-processable interface description (WSDL).
- Communication through messages (SOAP) using HTTP as transport protocol.
- XML serialization.

javax.jws.WebService;import javax.jws.WebMethod;import javax.jws.WebPar m;

```
@WebService(serviceName = "Calculator")public class CalculatorSayHelloService {
    /** This is a sample web service operation */@WebMethod(operationName = "add")
    public int add(@WebParam(name = "val1”) int val1, @WebParam(name = "val2”) intval2)
{
        return val1+val2;
    }
    @WebMethod(operationName = "add")
    public int sub(@WebParam(name = "val1”) int val1, @WebParam(name = "val2”) intval2)
{
        return val1-val2;
    }
    @WebMethod(operationName = "add")
    public int mul(@WebParam(name = "val1”) int val1, @WebParam(name = "val2”) intval2)
{
        return val1*val2;
    }
    @WebMethod(operationName = "add")
    public float div(@WebParam(name = "val1”) int val1, @WebParam(name = "val2”) int
val2) {
        return val1/val2;
    }}
```

XMLHttpRequest in AJAX with an example.

Ajax is an acronym for Asynchronous Javascript and XML. It is used to communicate with the server without refreshing the web page and thus increasing the user experience and better performance.

```
<!DOCTYPE html>
<html>
<body>
<div id="demo">
<h2>The XMLHttpRequest Object</h2>
<button type="button" onclick="loadDoc('ajax_info.txt', myFunction)">Change Content
</button>
</div>
<script>
function
  loadDoc(url,
  cFunction) {var
  xhttp;
  xhttp=new XMLHttpRequest();
  xhttp.onreadystatechange =
  function() {
    if (this.readyState == 4 &&
```

```
        this.status == 200) {
        cFunction(this);
      }
    };
    xhttp.open("G
    ET", url, true);
    xhttp.send();
  }
  function myFunction(xhttp) {
    document.getElementById("demo").
    innerHTML =xhttp.responseText;
  }
  </script>
  </body>
  </html>
```

AJAX program for

Establishing a connection between client server communication.

```
<asp:Button ID="Button6" runat="server" BackColor="Silver"
        onclientclick="store_memID()" style="margin-left: 20px" Text="Submit"
        Width="102px" Font-Bold="True" Height="28px" />


<script type = "text/javascript" src ="http://code.jquery.com/jquery-1.9.0.min.js"> </script>
 <script type = "text/javascript">


      // Function to caputure client-input- Member_ID.
      function store_memID() {
        // 'TextBox3' is the server side ID. To get the client side ID we do the following:-
        mem_ID = document.getElementById('<%=TextBox3.ClientID%>').value;
        //return confirm('TimeLine is displayed for: ' + mem_ID);
        ajax();
      }
      function ajax(){
        $.ajax({
          url: 'Default.aspx/MyMethod',
          type: 'POST',
          contentType: 'application/json; charset=utf-8',
          data: JSON.stringify({ memID: mem_ID }),
          success: function (result) {
          alert(result.d);
          }
        });
```

```
    }
    </script>
```

Server side:-

```
  public partial class _Default : System.Web.UI.Page
  {
    protected void Page_Load(object sender, EventArgs e)
    {}
   // Defining a page method.
    [WebMethod]
    public static string MyMethod(string mem_ID)
    {
      return string.Format("Thanks for calling me with id: " + mem_ID)
    }
```

Soap protocol in details.

Simple Object Access Protocol(SOAP) is a network protocol for exchanging structured data between nodes. It uses XML format to transfer messages. It works on top of application layer protocols like HTML and SMTP for notations and transmission. SOAP allows processes to communicate throughout platforms, languages and operating systems, since protocols like HTTP are already installed on all platforms.

SOAP was designed by Bob Atkinson, Don Box, Dave Winer, and Mohsen Al-Ghosein at Microsoft in 1998. SOAP was maintained by the XML Protocol Working Group of the World Wide Web Consortium until 2009.

Message Format:

SOAP message transmits some basic information as given below

Information about message structure and instructions on processing it.

Encoding instructions for application defined data types.

Information about Remote Procedure Calls and their responses.

The message in XML format contains three parts

Envelope:

It specifies that the XML message is a SOAP message. A SOAP message can be defined as an XML document containing header and body encapsulated in the envelope. The fault is within the body of the message.

Header:

This part is not mandatory. But when it is present it can provide crucial information about the applications.

Body:

It contains the actual message that is being transmitted. Fault is contained within the body tags.

Fault:

This section contains the status of the application and also contains errors in the application. This section is also optional. It should not appear more than once in a SOAP message.

Sample Message:

Content-Type: application/soap+xml

```
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
   <env:Header>
      <m:GetLastTradePrice xmlns:m="Some-URI" />
   </env:Header>
   <env:Body>
      <symbol xmlns:p="Some-URI" >DIS</symbol>
   </env:Body>
</env:Envelope>
```

Advantages of SOAP

SOAP is a light weight data interchange protocol because it is based on XML.

SOAP was designed to be OS and Platform independent.

It is built on top of HTTP which is installed in most systems.

It is suggested by W3 consortium which is like a governing body for the Web.

SOAP is mainly used for Web Services and Application Programming Interfaces (APIs).

Callback Functions In Ajax:

```
function start() {
   var xmlhttp = new XMLHttpRequest();
   var contentDiv = document.getElementById("Content");
   xmlhttp.open("POST", "Demo", true);
   xmlhttp.onreadystatechange=function() {
      if (xmlhttp.readyState==4 && xmlhttp.status==200) {
         console.log(xmlhttp.responseText);
      }
   }
   xmlhttp.setRequestHeader("Content-Type","application/x-www-form-urlencoded;charset=UTF-8");
```

```
        xmlhttp.send("FirstName=Nat&LastName=Dunn");
    }
    function start() {
        var xmlhttp = new XMLHttpRequest();
        var contentDiv = document.getElementById("Content");
        xmlhttp.open("POST", "Demo", true);
        xmlhttp.onreadystatechange=myCallBack;
        xmlhttp.setRequestHeader("Content-Type","application/x-www-form-
urlencoded;charset=UTF-8");
        xmlhttp.send("FirstName=Nat&LastName=Dunn");
    }
    function start() {
        var xmlhttp = new XMLHttpRequest();
        var contentDiv = document.getElementById("Content");
        xmlhttp.open("POST", "Demo", true);
        xmlhttp.onreadystatechange=function() {
          myCallBack(xmlhttp);
        };
        xmlhttp.setRequestHeader("Content-Type","application/x-www-form-
urlencoded;charset=UTF-8");
        xmlhttp.send("FirstName=Nat&LastName=Dunn");
```

FACULTY INCHARGE **HOD**